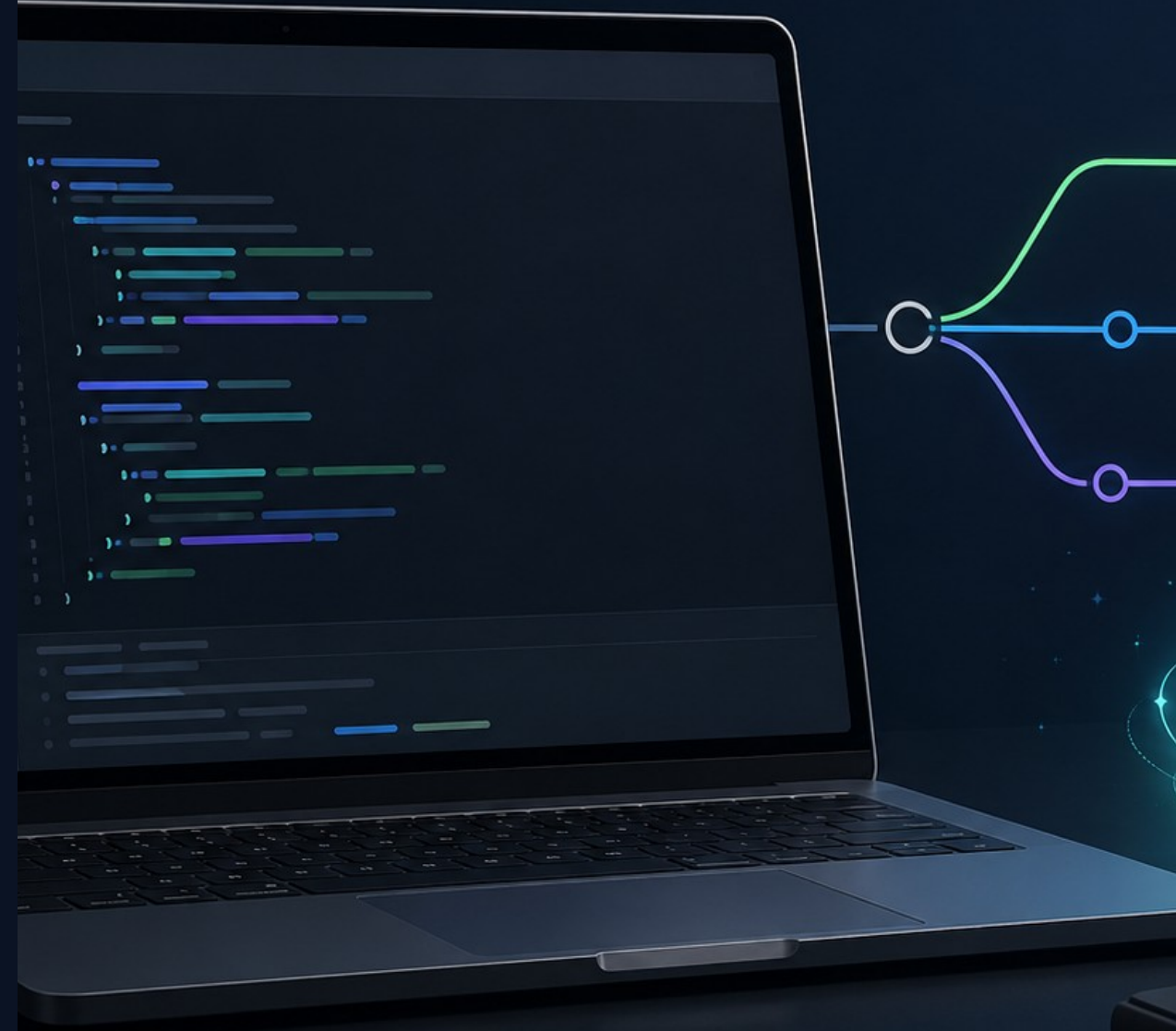


Codex

A practical intro for IsomerAPI

Offline reading deck: Windows app, CLI, Git workflow,
and safe first tasks for a Qt/C++ project

For plane reading - no internet required once downloaded



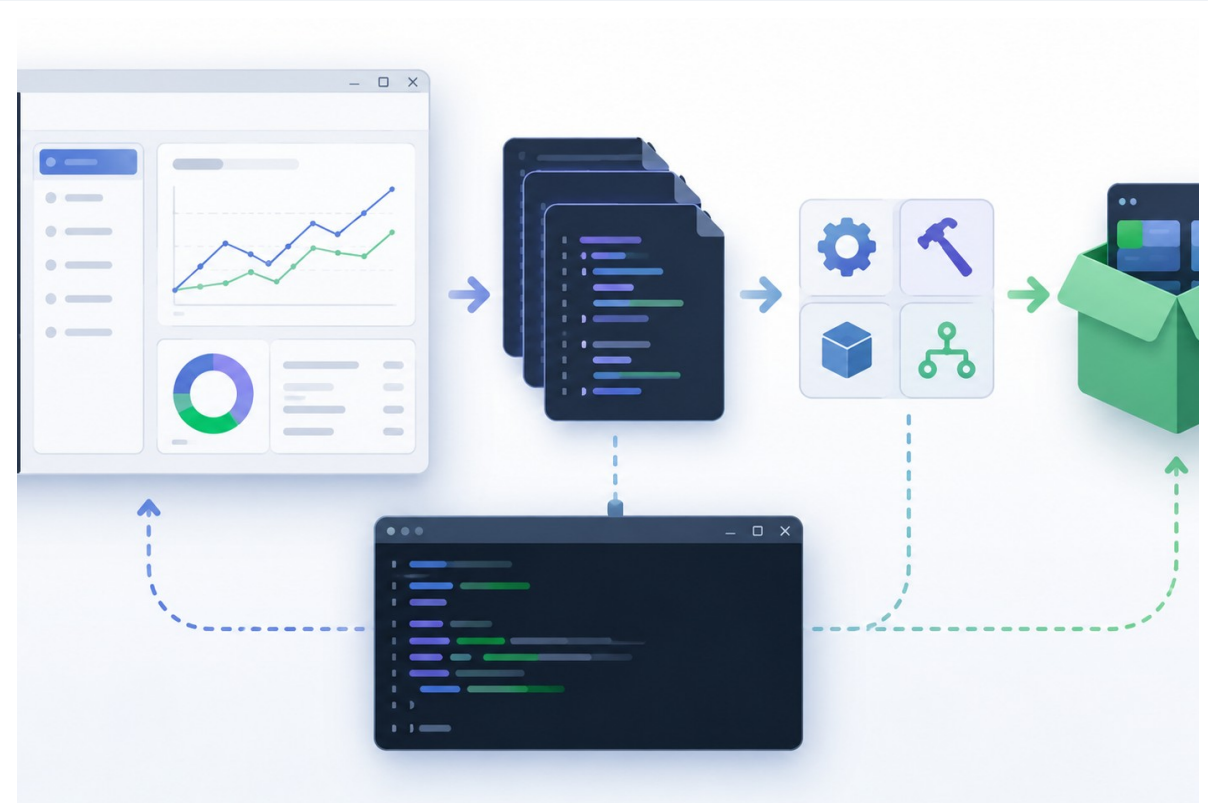
What Codex is

A coding agent that can inspect a repository, edit files, run commands, and prepare changes for review.

Think of Codex as a careful junior developer working inside your repo
It is strongest when the task is narrow:
cleanup, build notes, small bug fixes, tests, or refactoring
It should not silently redesign physics logic, database schema, or LISE++ integration without your review

Best mental model

You set constraints. Codex proposes a diff. You compile and approve.



Which Codex interface to use

For IsomerAPI on Windows 10, start with the desktop app; keep Qt Creator as the build authority.

Codex app

Best first choice on Windows: open the repo, run parallel threads, review diffs, use Git/worktrees.

Codex CLI

Terminal workflow from Git Bash or PowerShell. Good for repo inspection and command-line checks.

IDE extension

Useful later in VS Code/Cursor/Windsurf. Less central if Qt Creator stays your main IDE.

Use Codex to edit and prepare branches

Use Qt Creator/MSVC/MinGW to compile the real application

Use GitHub PRs for review before merging

First setup on Windows

A small checklist before letting Codex touch the repository.

Install/open the Codex Windows app and sign in
Open C:\IsomerAPI or a fresh clone of LISEcute/IsomerAPI
Make sure Git works: git status, git branch, git remote -v
Build once manually in Qt Creator to establish the baseline
Ask Codex to inspect only - no edits - for the first run

First prompt:

Inspect this Qt/C++ repository. Do not change files. Tell me the repo structure, build assumptions, accidental committed files, and safest first cleanup tasks.

Rule zero

Before accepting changes: review diff, compile in Qt Creator, then commit/push.

Safe Git workflow

Small branch, small diff, easy rollback. This is how to avoid a “helpful disaster.”



Start from a clean working tree: git status should be boring

Create a branch: `git switch -c cleanup/codex-first-pass`

Let Codex make a narrow diff only

Review every file, especially .pro, .cpp, database files, and paths

Compile manually; only then merge or create a PR

Never skip

The diff is the contract. If the diff is too big, stop and split the task.

Good first Codex task for IsomerAPI

Start with repository hygiene, not physics or GUI logic.

Prompt to Codex:

This is a Qt/C++ project named IsomerAPI intended to be integrated with LISE++.

Prepare a small cleanup branch that only:

1. removes README.md merge-conflict markers,
 2. removes IsomerAPI.pro.user from version control,
 3. checks .gitignore for QtCreator/MinGW/MSVC files,
 4. adds a short BUILD.md for QtCreator 16+, Qt 6.7.3, MinGW/MSVC.
- Do not modify source-code logic. Show me the diff before committing.

Why this is ideal

It tests Codex on Git, text edits, project structure, and build documentation without risking application behavior.

Expected files

README.md, .gitignore, BUILD.md, and Git tracking change for IsomerAPI.pro.user.

Your verification

Open Qt Creator and build both MinGW and MSVC versions if possible.

AGENTS.md: give Codex project rules

Codex reads project instructions before working. Use this to preserve your workflow.

Describe the project: Qt/C++ GUI for isomer levels and gamma lines

List build systems: Qt Creator 16+, Qt 6.7.3, MinGW and MSVC kits

State “do not modify physics/database logic unless explicitly requested”

Document verification: build in Qt Creator, inspect diffs, avoid user files

Minimal AGENTS.md:

IsomerAPI instructions

- Qt/C++ GUI project for LISE++ integration.
- Do not change physics logic, database schema, or file formats unless explicitly requested.
- Prefer small, reviewable commits.
- Never commit *.pro, user, build folders, .pdb, .bak, or local cert files.
- Verify by building in Qt Creator with Qt 6.7.3 MinGW MSVC.

Value

This turns repeated warnings into persistent repo-level instructions.

Qt/C++ build information Codex needs

Codex is not Qt Creator. Give it the map so it does not guess.

Project file

IsomerAPI.pro - qmake project configuration, sources, headers, resources, install paths.

Build outputs

_install for MinGW, _install_MSVC for MSVC, plus any local build-* directories.

Deployment

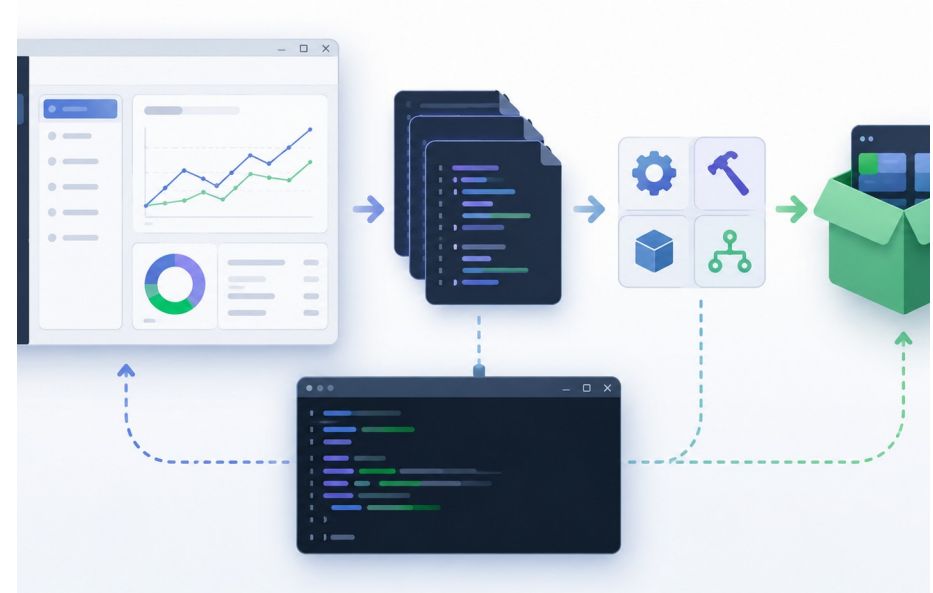
windeployqt.bat scripts copy Qt runtime DLLs into the install folder.

Tell Codex which kit is active before asking for compile fixes
Paste build errors from Qt Creator exactly; do not paraphrase them too much
Ask for minimal changes and require explanation of each changed file

Tasks where Codex should help immediately

These are low-risk and high-payoff for the repository.

- Clean README, BUILD.md, release notes, and comments
- Improve .gitignore for Qt Creator and Windows artifacts
- Find accidental binaries or user settings in Git
- Create deployment scripts for MinGW/MSVC
- Summarize code structure and dependencies
- Write small helper functions or refactor duplicate UI code after review



Good prompt phrase

"Make the smallest safe change that solves this, and show the diff."

Tasks to avoid at the beginning

Codex is powerful, but “large smart rewrite” is where trouble starts.

Do not ask it to redesign the isomer/gamma database model yet

Do not ask it to integrate deeply into LISE++ without a written design first

Do not accept large automatic rewrites of working Qt widgets

Do not let it change file formats, paths, or physics assumptions silently

Do not merge changes that were not compiled and tested locally

My recommended boundary

ChatGPT for design discussion; Codex for implementation of a narrow approved change.

If Codex proposes too much

Say: “Stop. Split this into three smaller PRs and start with documentation only.”

Review checklist before accepting a Codex diff

Fast is good. Fast plus review is better.

Does git diff show only the files you asked to change?
Did Codex remove local/user files rather than add more?
Did it preserve comments, paths, encodings, and Qt project settings?
Does the application build in the intended Qt kit?
Did windeployqt still package the executable correctly?
Can you explain every change in one sentence?

Useful command set:

```
git status
git diff --stat
git diff
git check-ignore -v IsomerAPI\pro.user
```

Then build in QtCreator.

Merge criterion

The code builds, the diff is small, and the purpose is clear.

Prompt cards for the plane

Copy these later when you are back online.

Inspect only:

Tell me how this Qt/C++ project is organized. Do not modify files. Identify likely build scripts, generated files, and risky tracked artifacts.

Build error:

Here is the exact Qt Creator build error. Explain the likely cause and propose the smallest change. Do not edit until I approve.

Small cleanup:

Create a branch and update documentation only. Do not touch source-code logic. Show the diff before committing.

Code review :

Review this diff for accidental behavior changes, Windows path problems, Qt resource issues, and files that should not be committed.

Style that works

Clear scope + "do not modify X" + "show diff" + "smallest safe change."

Recommended first week workflow

Use Codex repeatedly, but keep each pass boring and reversible.

Day 1: inspect repo, clean README, remove .pro.user, add BUILD.md

Day 2: add AGENTS.md and verify MinGW/MSVC deployment scripts

Day 3: ask Codex to summarize major classes and dependencies

Day 4: pick one small UI or build warning cleanup

Day 5: review whether Codex is reliable enough for larger tasks

For IsomerAPI

Start with housekeeping. Then use Codex for targeted implementation after we define the design here.

For LISE++ integration

Write the interface contract first. Then ask Codex to implement only that contract.

Sources and offline note

This deck is a compact plane-reading version of the official OpenAI Codex docs.

Codex overview: openai.com/codex

Codex app docs: developers.openai.com/codex/app

Windows app docs: developers.openai.com/codex/app/windows

Quickstart: developers.openai.com/codex/quickstart

CLI docs: developers.openai.com/codex/cli

AGENTS.md: developers.openai.com/codex/guides/agents-md

Best practices: developers.openai.com/codex/learn/best-practices

Plane reading tip

Use the PDF for reading. Use the PPTX later if you want to edit the deck for students or collaborators.